

T.C.
SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

BSM 401 BİLGİSAYAR MÜHENDİSLİĞİ TASARIMI

Hikâye Tabanlı 2D Piksel Oyun: Halisler

B221210096 – Hasan Yasir ARSLAN
G221210055 – Yavuz Selim ATEŞ

Bölüm : **BİLGİSAYAR MÜHENDİSLİĞİ**
Danışman : **Dr. Öğr. Üyesi Kayhan Ayar**

2025-2026 Güz Dönemi

ÖNSÖZ

Küresel oyun pazarındaki büyüme ivmesine rağmen, mevcut ekosistemde kısa ömürlü ve hızlı tüketime dayalı üretim modelleri baskın durumdadır. Finansal sürdürülebilirlik kaygısıyla tercih edilen düşük maliyetli yapımlar, nitelikli geliştirici gücünün, kültürel hikâye anlatıcılığı potansiyelinin yeterince değerlendirilememesine neden olmaktadır. Sektördeki derinlikli anlatı ve yenilikçi teknoloji eksikliği, oyuncu sadakatini ve katma değeri sınırlayan temel faktörler arasındadır. Standart diyalog ağaçlarının ötesine geçen Üretken Yapay Zekâ entegrasyonları, etkileşimi statik döngülerden kurtararak dinamik bir yapıya kavuşturmaktadır. Teknik inovasyon ve özgün hikâye kurgusunun birleşimi, pazarın ihtiyaç duyduğu uzun soluklu ve nitelikli üretim standartlarına ulaşılmasında zorunluluk arz etmektedir.

İÇİNDEKİLER

ÖNSÖZ.....	ii
İÇİNDEKİLER.....	iii
ŞEKİLLER LİSTESİ.....	v
ÖZET.....	vi

BÖLÜM 1.

GİRİŞ.....	1
1.1. Projeye Genel Bakış Ve Motivasyon.....	1
1.2. Oyunun Evreni Ve Hikaye Kurgusu.....	2
1.2.1. Temel oynanış mekanikleri.....	3
1.2.2. Karakterler	3
1.3. Benzer Oyunlar	4
1.4. İş Gelir Modeli ve Pazar.....	5

BÖLÜM 2.

OYUN TASARIMI VE KAVRAMSAL MODELLEME	7
2.1. Temel Oyun Döngüsü ve Mekanikler	7
2.1.1. Savaş mekanikleri ve matematiksel modeller	8
2.2. Bölüm Tasarımı ve İlerleme Haritası	9
2.3. Kullanıcı Arayüzü Tasarım Teorisi	10
2.4. Çevre Tasarımı ve Görsel Estetik.....	11
2.4.1. Özel sıralama eksen (Custom axis sorting).....	11
2.4.2. Pivot noktası optimizasyonu ve oklüzyon	12

BÖLÜM 3.

YAZILIM MİMARİSİ VE İSTEMCİ TARAFLI GELİŞTİRME	14
3.1. UML Tasarımı ve Sınıf Hiyerarşisi	14

3.2. Fizik Mimarisi ve Karakter Kontrolcüsü.....	15
3.3. Animasyon Sistemleri ve Karakter Kontrolcüsü (FSM)	17
3.3.1. Sprite entegrasyonu ve konfigürasyon	17
3.3.2. Animator kontrolcüsü ve mantıksal geçişler.....	18
3.4. Arayüz (UI) Programlama ve Prototipleme.....	19
3.4.1. UI altyapısı ve teknoloji seçimi.....	19
3.4.2. Dinamik diyalog ve etkileşim arayüzü	20
3.5. Oyun Akış Yönetimi (Cinematic Manager).....	21
BÖLÜM 4.	
YAPAY ZEKÂ ENTEGRASYONU VE DİNAMİK ETKİLEŞİM SİSTEMLERİ .	23
4.1. AI Model Seçimi ve Persona Mühendisliği.....	23
4.2. Modüler Npc Mimarisi	26
4.2.1. Veri kapsayıcıları	26
4.2.2. Sahne içi kimlik yönetimi	27
4.3. REST API İletişim Katmanı ve Veri İşleme	29
4.3.1. API mimarisi	30
4.3.2. Optimizasyon Süreci.....	30
4.3.3. JSON veri işleme.....	31
4.4. Diyalog Yönetimi ve Doğal Dil İşleme.....	31
4.4.1. Diyalog arayüzü yöneticisi	32
4.4.2. Kullanıcı girdisinin işlenmesi	32
BÖLÜM 5.	
SONUÇLAR VE ÖNERİLER	34
5.1. Proje Çıktılarının Değerlendirilmesi	34
5.2. Teknik Performans ve API Analizi.....	34
KAYNAKLAR.....	36
ÖZGEÇMİŞ.....	37

ŞEKİLLER LİSTESİ

Şekil 1.1.	Halisler Projesi Konsept Tasarım Görseli	2
Şekil 1.2.	Stardew Valley - Görsel Tasarım ve Kamera Bakış Açısı Referansı.....	5
Şekil 1.3.	'Halisler' Projesi Pazar Büyüklüğü Analizi	6
Şekil 2.1.	Halisler Projesi Temel Oyun Döngüsü (Core Game Loop).....	8
Şekil 2.2.	Halisler Projesi Mekânsal Kurgusu	9
Şekil 2.3.	Oyun Arayüzü Sınıflandırma Matrisi.....	11
Şekil 2.4.	Unity Editör üzerinde Transparency Sort Mode ayarlarının yapılandırılması.....	12
Şekil 2.5.	Karakterin bir nesnenin arkasına geçtiği andaki derinlik testi.....	13
Şekil 3.1.	Halisler Projesi UML Sınıf Mimarisi.....	15
Şekil 3.2.	PlayerController sınıfı içerisindeki vektörel hareket hesaplaması.....	17
Şekil 3.3.	Doku ayarları optimize edilmeden önce ve sonra.....	18
Şekil 3.4.	Karakterin Idle ve Run durumları arasındaki geçiş mantığını gösteren Animator şeması.....	19
Şekil 3.5.	UI Tasarımı Prototipi.....	21
Şekil 3.6.	IEnumerator PlayIntroSequence fonksiyonu.....	22
Şekil 4.1.	GeminiManager sınıfı ve veriyi API'ye gönderen SendRequest fonksiyonu.....	25
Şekil 4.2.	NpcProfile veri yapısının Unity Inspector üzerindeki görünümü.....	27
Şekil 4.3.	NPC'ye profilini yükleyen ve duruma göre cevap üretmesini sağlayan Talk fonksiyonu.....	29
Şekil 4.4.	API testlerinde kota sınırının aşılmasıyla oluşan hata durumu.....	31

Şekil 4.5.	Diyalog Arayüzü Yöneticisinin çalışma anı.....	32
Şekil 4.6.	Kullanıcı girdisine ("Adın ne asker?") karşılık üretilen, karakter kimliğine uygun dinamik yapay zekâ yanıtı.....	33
Şekil 5.1.	Bağlam kaybı nedeniyle NPC'nin oyuncuyu tanıyamaması ve diyalog bütünlüğünün bozulması.....	35

ÖZET

Anahtar Kelimeler: Oyun Geliştirme, Üretken Yapay Zekâ, Doğal Dil İşleme, REST API Mimarisi, Dinamik NPC

Hızlı tüketime dayalı üretim modellerinin baskın olduğu oyun sektöründe, nitelikli hikâye anlatımı ve oyuncu etkileşiminin teknik inovasyonlarla derinleştirilmesi önemli bir ihtiyaç haline gelmiştir. Bu tasarım çalışmasında, geleneksel 2D piksel sanat estetiği ile modern Üretken Yapay Zekâ teknolojilerini birleştiren, aksiyon-macera türünde hibrit bir oyun mimarisi geliştirilmiştir. Proje kapsamında, Büyük Dil Modelleri (LLM) Unity oyun motoruna entegre edilerek, standart ve statik diyalog ağaçları yerine bağlama duyarlı, dinamik yanıt üreten ve oyuncu ile doğal dilde iletişim kurabilen bir NPC sistemi tasarlanmıştır.

Teknik altyapı olarak Nesne Yönelimli Programlama prensipleri benimsenmiş, Google Gemini servisi ile oyun motoru arasında asenkron veri akışı sağlayan bir REST API katmanı oluşturulmuştur. Ayrıca, özel sıralama eksenleri ve durum makineleri kullanılarak 2D düzlemde görsel derinlik ve akıcı mekanikler sağlanmıştır. Elde edilen sonuçlar, genel amaçlı dil modellerinin oyun içi etkileşimde teknik olarak uygulanabilir olduğunu, ancak ticari akıcılık ve performans için optimize edilmiş modellere ihtiyaç duyulduğunu ortaya koymuştur.

BÖLÜM 1. GİRİŞ

Bu bölümde, oyun sektöründeki hızlı tüketime dayalı üretim modellerine nitelikli bir alternatif olarak geliştirilen projenin temel vizyonu ve kapsamı sunulmaktadır. Çalışmanın çıkış noktasını oluşturan oyuncu beklentileri ve sektörel analizlerden yola çıkılarak; projenin kurgusal evreni, hikâye yapısı ve teknik oyun mekanikleri detaylandırılmıştır. Ayrıca projenin hedef kitlesi, pazar potansiyeli, gelir modeli ve geliştirme sürecinde kullanılan yazılım teknolojileri incelenerek, çalışmanın özgün değeri ve teknik çerçevesi ortaya konulmuştur.

1.1. Projeye Genel Bakış Ve Motivasyon

Günümüz dijital pazarında hâkim olan hızlı tüketilen (hyper-casual) yapımların aksine, oyuncunun projeye olan bağlılığını sağlamak, yüzeysel mekaniklerden çok daha kapsamlı bir tasarım mimarisini zorunlu kılmaktadır. Bu bağlamda projenin temel hedefi; oyuncuyu pasif bir tüketici konumundan çıkarıp, oyun dünyasının aktif ve yaşayan bir bileşeni haline getirmektir. Bu etkileşim mimarisi; hikâye kurgusundaki bilinmezliklerle beslenen "merak", kültürel kodlarla zenginleştirilmiş karakter profilleriyle sağlanan "aidiyet" ve tatmin edici mekaniklerle oluşan "akış" dinamikleri üzerine inşa edilmiştir. "Halisler", bu üç temel unsuru teknik zeminde birleştirerek kısa vadeli bir etkileşim yerine, sürükleyici ve dinamik bir deneyim sunmayı amaçlamaktadır.

2D piksel sanat estetiğine sahip "Halisler"; yapay zekâ destekli dinamik anlatısı, Türk kültürüyle harmanlanmış karakter tasarımları ve derin hikâye kurgusuyla sektörel eksikliklere çözüm niteliği taşıyan uzun soluklu bir macera oyunudur. Projenin çözüm önerisi üç ana teknik sütun üzerinde yükselmektedir. İlk olarak, "Yapay Zekâ Destekli Dinamik Evren" yaklaşımıyla standart diyalog döngüleri yerine Üretken Yapay Zekâ entegrasyonu benimsenmiş, böylece NPC'lerin oyuncu etkileşimlerine bağlamsal ve dinamik tepkiler vermesi sağlanarak simülasyonun

inandırıcılığı artırılmıştır. Buna ek olarak, "Kültürel Temsil ve Rol Model Oluşturma" hedefi doğrultusunda Türk toplum yapısı ve kültürel mirası modern bir estetikle dijital ortama aktarılmış, oyuncuların oyun evreniyle kurduğu duygusal bağın güçlendirilmesi amaçlanmıştır. Son olarak, "Derin ve Sürükleyici Anlatı" kurgusu sayesinde nitelikli sinematik ara sahneler ve stratejik oyun mekanikleri entegre edilerek, oyuncunun odağını sürekli canlı tutan bir yapı oluşturulmuştur.

Sonuç olarak bu çalışma; yenilikçi teknolojilerin ve özgün kültürel değerlerin entegrasyonu, yerli oyun sektöründe hem teknik inovasyon hem de finansal sürdürülebilirlik açısından başarılı bir model oluşturma vizyonunu taşımaktadır.



Şekil 1.1. Halisler Projesi Konsept Tasarım Görseli

1.2. Oyunun Evreni Ve Hikaye Kurgusu

"Halisler" projesi, büyücüler ve Orionlar gibi çeşitli fantastik varlıkların bir arada yaşadığı, olağanüstü unsurlarla bezeli kurgusal bir evrende geçmektedir. Oyunun temel anlatı yapısı, belirli bir bölgeyle tüm iletişimin kopması; bölgeye sevk edilen diplomatik elçilerin, ticari kervanların ve askeri birliklerin akıbetinden haber alınamaması gizemi üzerine kurgulanmıştır. Söz konusu belirsizliği aydınlatmak amacıyla bölgeye yönelik bir keşif ve araştırma göreviyle yetkilendirilen ana karakterin hazırlık süreci, bulunduğu şehre düzenlenen ani ve yıkıcı bir saldırıyla

kesintiye uğrar. Hikâyedeki bu kırılma noktası, senaryoyu salt bir araştırma görevinden çıkararak oyuncuyu hem hayatta kalma mücadelesine hem de kaosu ardındaki köklü sınırları çözmeye iten aksiyon odaklı derin bir olay örgüsüne dönüştürmektedir.

1.2.1. Temel oynanış mekanikleri

"Halisler" projesinin oynanış mekanikleri, oyuncuyu sürekli "akış" durumunda tutmayı hedefleyen, rastgele tuş girişine dayalı yüzeysel yapıdan uzak, stratejik derinlik ve zamanlama hassasiyeti barındıran deterministik bir model üzerine inşa edilmiştir. Ana karakterin yetenek seti, büyü kullanımı ve savunma stratejileri olmak üzere iki temel eksenle özelleştirilmiştir. İlk eksen olan "Kombo Tabanlı Büyü Sistemi"nde, yeteneklerin tek bir tuş ataması yerine spesifik kombinasyonlarla tetiklenmesi öngörülmüş; bu sayede büyü kullanımı, ezber ve refleksin birleştiği bilişsel bir sürece dönüştürülmüştür.

Mekanik tasarımın ikinci eksenini olan "Düşünsel Savunma ve Savuşturma (Parry)" sistemi ise, pasif savunma anlayışını reddederek oyuncuyu aktif karar alma sürecine yönlendirmektedir. Yüksek risk-yüksek ödül dengesi üzerine kurulu bu yapıda, saldırı anındaki milisaniyelik doğru zamanlama düşmanı etkisiz hale getirip karşı saldırı fırsatı yaratırken, zamanlama hatası savunma zafiyetiyle cezalandırılır. Bu yaklaşım, savunma eylemini statik bir durumdan çıkararak, düşman hareketlerinin analiz edildiği stratejik bir mücadele döngüsüne dönüştürmektedir.

1.2.2. Karakterler

"Halisler" projesinin karakter tasarım mimarisi, oyuncunun evrene duyduğu aidiyet hissi ile projenin temel misyonu olan kültürel temsil hedefinin kesişim noktasında kurgulanmıştır. Kültürel derinliğin teknik inovasyonla entegrasyonunu amaçlayan bu yapı, temelini özgün karakterlerin oluşturduğu yaşayan bir simülasyon evreni sunmayı hedefler. Türk toplumsal yapısını ve ahlaki değerlerini yansıtan, genç oyuncular için birer rol model niteliği taşıyan bu derinlikli profiller; yabancı

kültürlerin domine ettiği içeriklere yerli bir alternatif oluşturarak, oyuncunun kendi değer yargılarıyla bağ kurabileceği bir etkileşim zemini yaratmaktadır.

Söz konusu kültürel otantikliğin teknik altyapıya taşınması sürecinde, geleneksel oyun diyalog sistemlerinin sınırları aşılarak "Karakter Geçmişi ve Kişilik Özellikleri ile Eğitilmiş Üretken Yapay Zekâ Tabanlı Dinamik NPC" mimarisi benimsenmiştir. Bu doğrultuda, her bir NPC için kurgulanan özgün karakter geçmişleri ve kişilik profilleri yapay zekâ entegrasyonu ile işlenmiş; böylece oyunculara statik menü seçimleri yerine serbest metin formatında doğal diyalog kurma imkânı tanınmıştır. NPC'lerin tanımlı kişilik profilleri ve ahlaki kodlar ışığında ürettiği dinamik ve bağlama uygun yanıtlar, her oyuncuya özgü benzersiz bir deneyim sunarak hedeflenen aidiyet duygusunu teknik zeminde güçlendirmektedir.

1.3. Benzer Oyunlar

"Halisler" projesi, konvansiyonel tür sınıflandırmalarının ötesine geçerek, pazarda başarısı kanıtlanmış farklı tasarım paradigmalarını sentezleyen hibrit bir mimari üzerine inşa edilmiştir. Görsel tasarım ve perspektif kurgusu bağlamında Stardew Valley yapıtını referans alan projede, 2D piksel sanat estetiği ve "Top-Down" (Kuş Bakışı) kamera açısı benimsenmiştir [1]. Minimalist bir görsel dilin, derinlikli ve yaşayan bir simülasyon evreni kurgulayarak oyuncuda aidiyet hissi oluşturmadaki başarısı, projenin görsel tasarım felsefesinin temel dayanağını oluşturmaktadır.



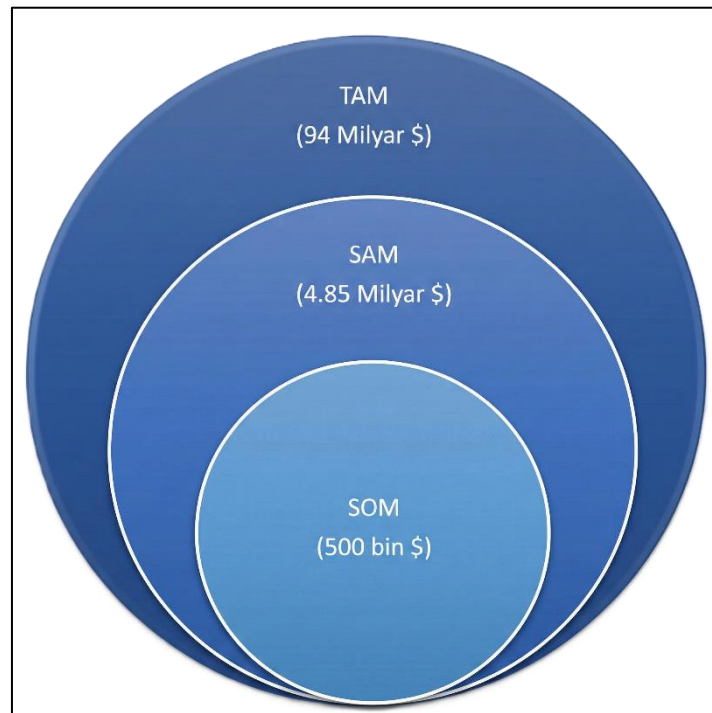
Şekil 1.2. Stardew Valley - Görsel Tasarım ve Kamera Bakış Açısı Referansı

Savaş mekanikleri ve oyun akışı bağlamında ise Hollow Knight, Dead Cells ve Ori serisi gibi yetenek tabanlı dövüş sistemleriyle öne çıkan yapımlar teknik referans olarak belirlenmiştir. Projede kurgulanan "düşünsel savunma" ve "Savuşturma (Parry)" mekanikleri, söz konusu yapımların sunduğu akıcı mücadele dinamikleri referans alınarak geliştirilmiştir. Bu çerçevede; Hollow Knight'ın zorlu 'boss' savaşları, Dead Cells'in akıcı kombo zincirleri ve Ori serisinin çevik hareket kabiliyetleri, hedeflenen tatmin edici savaş döngüsü için temel dayanak noktalarını oluşturmaktadır. Proje mimarisi, endüstride rüştünü ispat etmiş bu mekaniksel unsurları, "Üretken Yapay Zekâ Tabanlı Dinamik Diyalog Sistemi" gibi yenilikçi teknolojilerle sentezleyerek pazarda özgün ve rekabetçi bir konumlama elde etmeyi amaçlamaktadır.

1.4. İş Gelir Modeli ve Pazar

Projenin odaklandığı tüketici profili; 12 yaş ve üzeri, aksiyon-macera türüne ilgi duyan ve anlatı derinliğine sahip 2D yapımları tercih eden kullanıcılardır. Ticari yapılanma, son kullanıcıya doğrudan erişimi hedefleyen B2C modeli üzerine inşa edilmiş olup; gelir akışı, dijital dağıtım ağları üzerinden gerçekleştirilecek bölgesel fiyatlandırma stratejisine dayalı tek seferlik premium satış modeline dayanmaktadır.

Küresel ölçekte bağımsız oyun pazarının sergilediği yıllık yüzde 7-9 oranındaki büyüme ivmesi ve 2026 yılına kadar 3 milyar dolar hacmi aşacağı yönündeki projeksiyonlar, sektörün genişleme potansiyelini ortaya koymaktadır [2], [3]. Buna karşın Türkiye ekosisteminin, özellikle derin anlatılı PC ve konsol projelerinde yeterli pazar payına sahip olmaması, proje için stratejik bir fırsat alanı olarak değerlendirilmektedir. Pazar analizi verileri ışığında, 94 milyar dolarlık Toplam Adreslenebilir Pazar (TAM) içerisinde, bağımsız yapımlara ayrılan 4.85 milyar dolarlık Hizmet Verilebilir Pazar (SAM) dilimi öncelikli hedef olarak belirlenmiştir. Bu doğrultuda oluşturulan üç yıllık Ulaşılabilir Pazar (SOM) projeksiyonu ise 100.000 adetlik satış hacmi ve buna bağlı olarak 500.000 dolarlık ciro hedefini kapsamaktadır.



Şekil 1.3. 'Halisler' Projesi Pazar Büyüklüğü Analizi

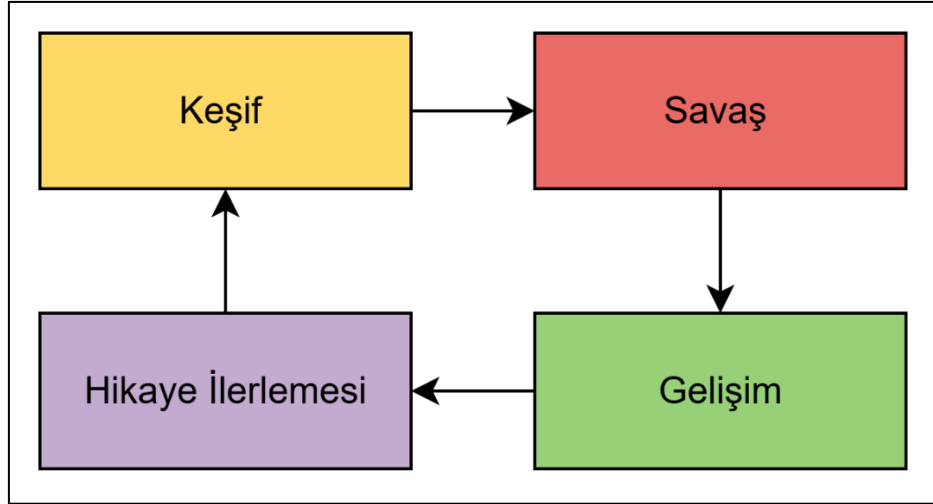
BÖLÜM 2. OYUN TASARIMI VE KAVRAMSAL MODELLEME

Bu bölümde, projenin teknik geliştirmesine geçilmeden önceki teorik tasarım süreci ve oyunun temel çalışma prensipleri detaylandırılmaktadır. Oyunun çekirdek mekaniğini oluşturan savaş ve savunma sistemlerinin matematiksel modelleri ile oyuncu deneyimini şekillendiren akış diyagramları bu kapsamda ele alınmıştır. Ayrıca, oyunun mekânsal kurgusunu belirleyen bölüm tasarımları, 2D görsel estetik (tilemap) tercihleri ve kullanıcı arayüzü (UI) teorisi incelenerek; oyunun kavramsal mimarisinin teknik uygulamaya nasıl hazırlandığı ortaya konulmuştur.

2.1. Temel Oyun Döngüsü ve Mekanikler

"Halisler" projesinin oynanış dinamikleri, oyuncunun anlatıdan kopmasını engelleyen ve deneyimi sürekli akış halinde tutan iç içe geçmiş döngüsel bir yapı üzerine inşa edilmiştir. Anlık reflekslere dayalı mikro etkileşimler, bu yapının temelini oluşturur. Oyuncunun düşman davranışlarını analiz ederek doğru zamanlama ile savunma (Parry) gerçekleştirmesi ve düşman üzerinde taktiksel avantaj sağlaması bu katmanda gerçekleşir. Benzer bir anlık geri bildirim mekanizması NPC etkileşimlerinde de kurgulanmış olup; oyuncunun eylemlerine verilen bağlamsal ve dinamik yanıtlar, hikâyedeki aktif rolünü pekiştirerek oturum süresince ilginin canlı kalmasını sağlar.

Bu anlık etkileşimler, uzun vadeli ilerleme motivasyonunu oluşturan makro döngü ile bütünleşerek oyunun genel yapısını tamamlar. Oyuncu; evreni keşfetme, stratejik mücadelelerle engelleri aşma ve elde ettiği deneyim puanlarıyla karakterini geliştirme süreçleri üzerinden derin bir anlatının parçası olduğunu hisseder. Keşif, savaş ve gelişim arasındaki bu organik bağ, merak unsurunu sürekli tetikleyerek oyuncuda "bir sonraki adım" motivasyonu yaratır ve projeye olan sadakati güçlendirir.



Şekil 2.1. Halisler Projesi Temel Oyun Döngüsü (Core Game Loop)

2.1.1. Savaş mekanikleri ve matematiksel modeller

"Halisler" projesinin savaş mekaniği, rastgele tuş basımına dayalı kaotik bir yapı yerine; zamanlama hassasiyeti ve stratejik karar alma süreçleri üzerine kurulu deterministik bir sistem olarak tasarlanmıştır. Özellikle oynanışın merkezinde yer alan savunma (Parry) mekaniği, oyuncunun tepki süresi ile oyun motorunun kare hızı arasındaki ilişkiyi temel alan matematiksel bir modellemeyle yönetilmektedir. Bu bağlamda bir saldırının başarıyla savuşturulabilmesi; oyuncu tarafından sağlanan savunma girdisinin (t_{input}), düşman saldırısının temas anına (t_{hit}) göre tanımlanan belirli bir tolerans penceresi (Δ_{window}) içerisinde gerçekleşmesi koşuluna bağlanmıştır. (Denklem 2.1)

$$Damage = \begin{cases} 0 & \text{if } |t_{hit} - t_{input}| \leq \Delta_{window} \\ BaseDamage & \text{if } |t_{hit} - t_{input}| > \Delta_{window} \end{cases} \quad (2.1)$$

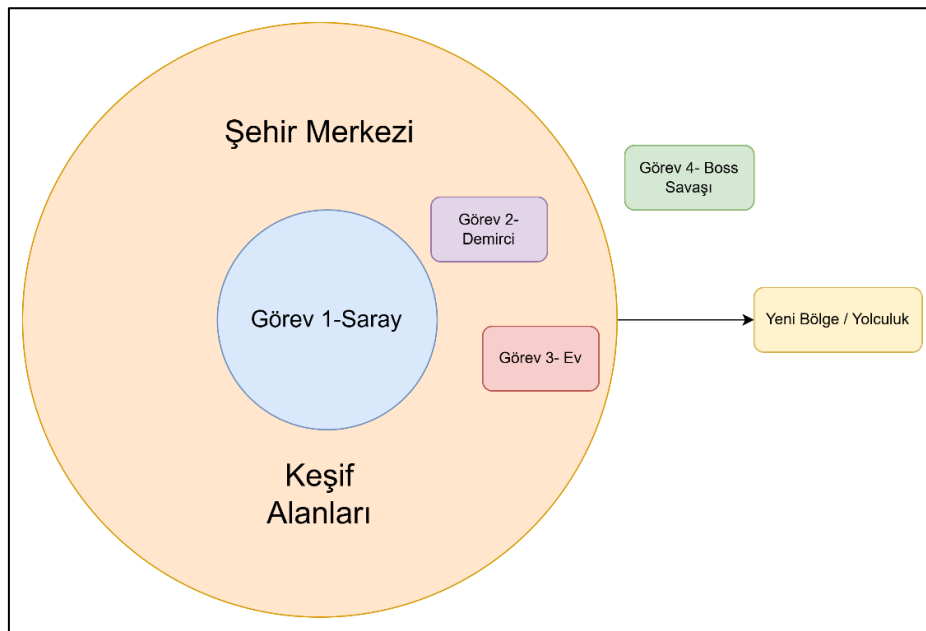
Tasarım mimarisinde (Δ_{window}) değeri 0.25 saniye olarak standardize edilmiştir. Oyuncunun bu zaman aralığını başarıyla yakalaması durumunda, sisteme yansıyan hasar değeri sıfırlanmakta ve saldırgan 'sersemleme' durumuna geçirilerek oyuncuya taktiksel avantaj sağlanmaktadır. Zamanlamanın tutturulamadığı senaryolarda ise karakter, düşmanın taban saldırı gücü (BaseDamage) oranında hasara maruz kalmaktadır. Oyuncuyu sürekli 'akış' kanalında tutmayı hedefleyen bu risk-ödül

mekanizmasının teknik implementasyonu, oyunun kare hızı dalgalanmalarından (FPS) etkilenmemesi ve deterministik yapının korunması adına motorun 'DeltaTime' fonksiyonu referans alınarak kurgulanmıştır.

2.2. Bölüm Tasarımı ve İlerleme Haritası

"Halisler" projesinin bölüm tasarımı, The Witcher [4] serisinin erken dönem örneklerinde gözlemlenen hikâye odaklı yarı-açık dünya (Story-Driven Semi-Open World) yaklaşımını esas almaktadır. Oyunun ilk perdesi, yaşayan ve etkileşime açık bir şehir yapısı etrafında şekillenir. Mekânsal kurgu, hiyerarşik olarak birbirini takip eden üç temel katman üzerine inşa edilmiştir.

Birinci katman olan "Şehir Merkezi"; oyuncunun serbestçe dolaşabildiği, yapay zekâ destekli karakterlerle etkileşime girdiği ve yan görevleri keşfettiği ana merkez niteliğindedir. Bu yapıyı, şehir sınırları içerisinde yer alan ancak ana hikâye ve aksiyon sekanslarının tetiklendiği özelleştirilmiş "Görev Noktaları" takip eder. Mekânsal kurgunun son halkası olan "Yolculuk" evresinde ise; şehirdeki hikâye düğümlerinin çözülmesini takiben karakter, anlatı gereği fiziksel bir yolculuğa çıkarak oyunu tamamen yeni bir haritaya ve atmosfere taşımaktadır.



Şekil 2.2. Halisler Projesi Mekânsal Kurgusu

2.3. Kullanıcı Arayüzü Tasarım Teorisi

"Halisler" projesinin arayüz mimarisi, oyuncunun oyun dünyasıyla kurduğu bağı zedelemekten gerekli teknik verileri sunmak amacıyla, literatürde kabul görmüş Fagerholt ve Lorentzon'un UI Sınıflandırma Matrisi referans alınarak kurgulanmıştır. Bu çerçevede tasarım, dört temel kategoride ele alınmıştır. İlk olarak, oyunun hikâyesine veya fiziksel dünyasına dahil olmayan ve doğrudan oyuncuya hitap eden Oyun Dışı (Non-Diegetic) öğeler kapsamında; ana menü, envanter ekranı ve ayarlar gibi akışı durduran pencereler, piksel sanat estetiğine uygun ve akıcı geçişlere sahip animasyonlu paneller olarak tasarlanmıştır.

Oyun dünyası içinde yer almakla birlikte hikâye örgüsüne dahil olmayan Mekânsal (Spatial) öğeler ise oyuncunun navigasyonunu kolaylaştırmak amacıyla karakter çevresine entegre edilmiştir. Tanıdık NPC'lerin üzerinde beliren isim etiketleri, görev hedeflerini işaret eden yön okları ve etkileşime girilebilir nesneler üzerindeki tuş ikonları (Örn: "E" tuşu) bu kategoride değerlendirilmiştir. Buna ek olarak, oyun dünyasında fiziksel karşılığı bulunmayan ancak karakterin durumuyla ilişkili soyut göstergeler Temsili (Meta) öğeler başlığı altında toplanmıştır. Örneğin, sağlık puanının kritik seviyeye düştüğü anlarda ekran kenarlarının kırmızılaşması ve kalp ritmine göre titremesi; karakterin acıyı hissetmesi ile oyuncunun bu durumu görsel bir metafor üzerinden algılaması arasındaki bağı kurmaktadır.

Son olarak hem oyuncunun hem de karakterin görebildiği ve oyun dünyasının doğal bir parçası olan Oyun İçi (Diegetic) öğeler, arayüz takibini minimize ederek görsel geri bildirimlere odaklanmıştır. Bir gösterge çubuğu kullanmak yerine; özel yetenek dolduğunda silah ucunda beliren mavi parıltı veya hasar alındığında karakterin topallamaya başlaması gibi fiziksel değişimler, oyuncuya gerekli bilgiyi organik bir şekilde aktarmaktadır [5].



Şekil 2.3. Oyun Arayüzü Sınıflandırma Matrisi

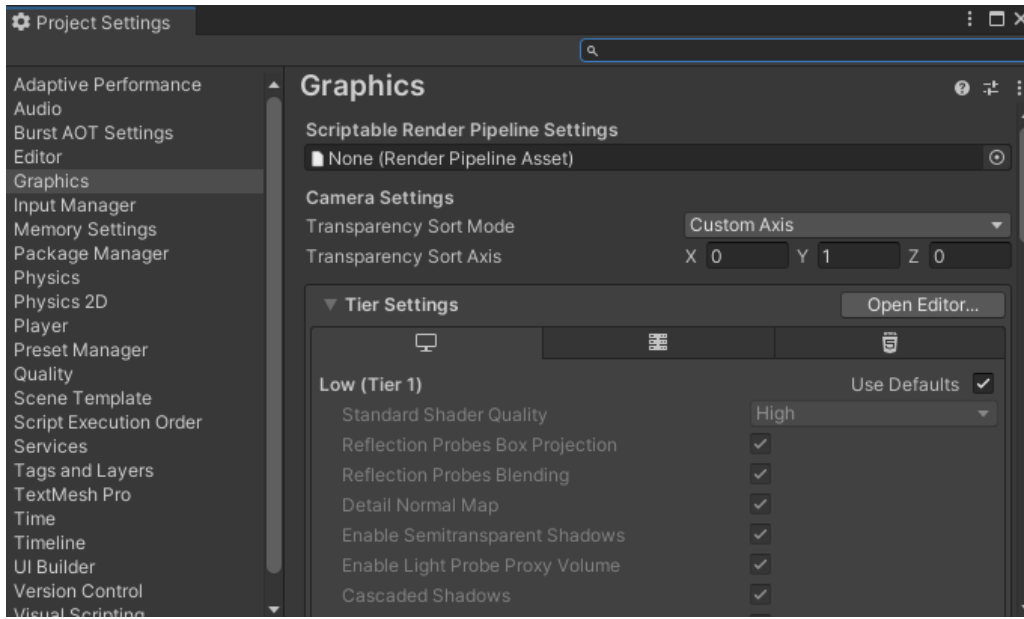
2.4. Çevre Tasarımı ve Görsel Estetik

"Halisler" projesinin harita tasarımı, Stardew Valley benzeri "Kuş Bakışı" (Top-Down) kamera açısı ve bu perspektife uygun piksel sanat estetiğine sahip varlıklarla kurgulanmıştır. İki boyutlu bir düzlem üzerinde çalışılmasına karşın, oyuncuya üç boyutlu bir derinlik ve hacim hissiyatının aktarılabilmesi, oyunun görsel inandırıcılığı açısından kritik bir önem arz etmektedir. Bu bağlamda, hedeflenen "Aidiyet" ve "Gerçekçilik" atmosferini teknik zeminde desteklemek amacıyla, Unity oyun motoru üzerinde özelleştirilmiş bir görsel sıralama (sorting) mühendisliği uygulanmış ve nesnelerin birbirleriyle olan derinlik ilişkileri optimize edilmiştir [6].

2.4.1. Özel sıralama eksen (Custom axis sorting)

Standart 2D render işlemlerinde nesnelerin çizim sırası genellikle kameraya olan uzaklıklarına (Z eksen) göre belirlenmektedir. Ancak projenin benimsediği bakış açısı gereği, derinlik algısının Y eksen (Dikey Konum) üzerinden kurgulanması zorunlu hale gelmiştir. Bu perspektif kuralına göre, ekranda "aşağıda" konumlanan bir nesne kameraya daha yakın, "yukarıda" duran bir nesne ise daha uzak olarak algılanmalıdır.

Söz konusu teknik gereksinimi karşılamak adına Unity Grafik Ayarları üzerinde yapılan yapılandırma ile "Transparency Sort Mode" varsayılan moddan "Custom Axis" moduna alınmış ve sıralama vektörü (X:0, Y:1, Z:0) olarak ayarlanmıştır. Bu konfigürasyon sayesinde render motoru, sahnedeki tüm sprite'ların Y eksenindeki pozisyonlarını anlık olarak karşılaştırarak, daha düşük Y değerine sahip objeleri diğerlerinin önüne çizecek şekilde optimize edilmiştir.



Şekil 2.4. Unity Editör üzerinde Transparency Sort Mode ayarlarının yapılandırılması.

2.4.2. Pivot noktası optimizasyonu ve oklüzyon

Özel sıralama eksenini ayarının (Custom Axis) doğru çalışabilmesi ve nesnelerin birbirini kusursuz şekilde perdeleyebilmesi (Occlusion) için varlıkların merkez noktalarının yeniden yapılandırılması gerekmiştir. Bu teknik gereksinim doğrultusunda; karakter ve çevre nesnelerinin Sprite Pivot noktaları, varsayılan "Merkez" (Center) konumundan, nesnenin zeminle temas ettiği "Alt Orta" (Bottom Center) noktasına taşınmıştır.

Gerçekleştirilen bu pivot optimizasyonu ile dinamik oklüzyon mekanizması işlevsel hale getirilmiştir. Bu sayede, karakter ekranda dikey olarak hareket edip bir nesneye yaklaştığında, pivot noktası ilgili nesnenin pivot noktasını aştığı an render motoru tarafından otomatik olarak arka plana çizilmektedir. Bu sistem, 2D düzlemdeki

perspektif hatalarını ortadan kaldırarak oyuncuya nesnelerin "etrafından dolaşma" hissini ve mekânsal derinliği teknik bir zeminde sunmaktadır.



Şekil 2.5. Karakterin bir nesnenin arkasına geçtiği andaki derinlik testi

BÖLÜM 3. YAZILIM MİMARİSİ VE İSTEMCİ TARAFLI GELİŞTİRME

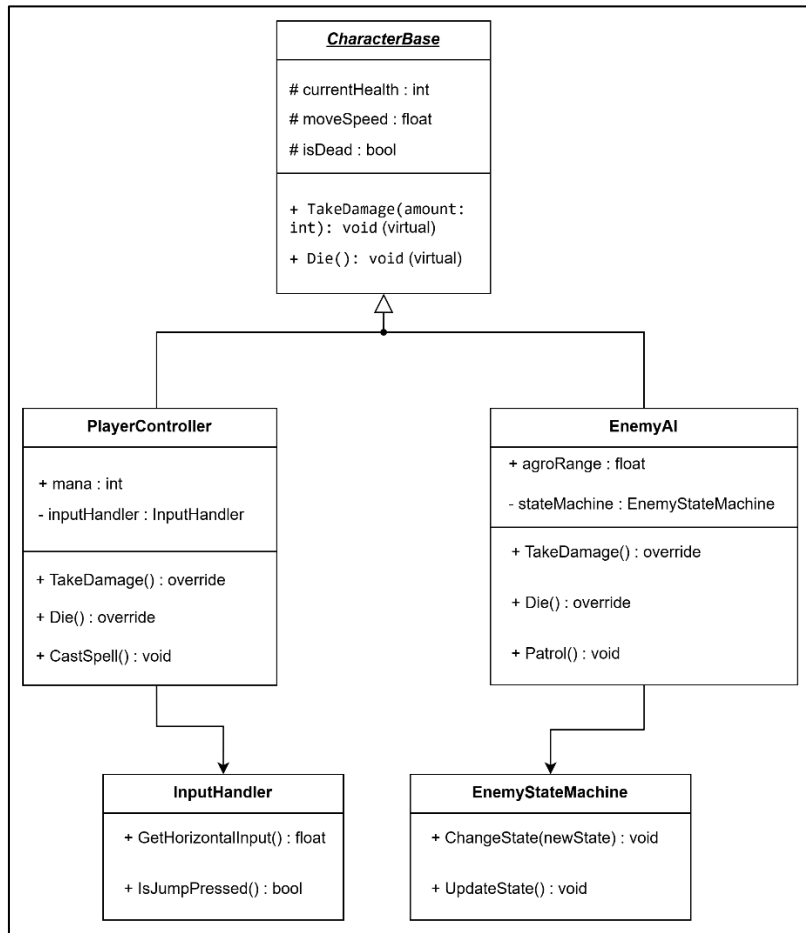
Bu bölümde, teorik tasarımın Unity oyun motoru üzerinde çalışan somut bir yazılım mimarisine dönüştürülme süreci detaylandırılmaktadır. Projenin sürdürülebilirliğini ve genişletilebilirliğini sağlamak amacıyla, Nesne Yönelimli Programlama (OOP) prensipleri çerçevesinde kurgulanan sınıf hiyerarşileri ve UML şemaları incelenmiştir. Ayrıca, vektör tabanlı karakter kontrolcülerinin implementasyonu, 2D fizik motoru entegrasyonu ve animasyon durum makinelerinin (FSM) teknik altyapısı ele alınmıştır. İstemci taraflı geliştirme süreçleri kapsamında, kullanıcı arayüzü programlaması ve oyun akışını yöneten temel algoritmalar da bu bölümün kapsamını oluşturmaktadır.

3.1. UML Tasarımı ve Sınıf Hiyerarşisi

"Halisler" projesinin teknik altyapısı kurgulanırken, sistemin ilerleyen süreçte genişletilebilir olması ve bakım kolaylığı temel öncelik olarak belirlenmiştir. Bu doğrultuda, Nesne Yönelimli Programlama (OOP) paradigması temel alınarak, SOLID prensipleri ve Kendini Tekrar Etme (DRY) yaklaşımı ile modüler bir mimari tasarlanmıştır. Oluşturulan sınıf hiyerarşisi ilk olarak soyutlama ve kalıtım prensipleri çerçevesinde şekillendirilmiş; oyun dünyasındaki oyuncu, düşmanlar ve NPC'ler gibi tüm etkileşimli canlı varlıkların sağlık puanı ve hareket hızı gibi ortak özellikleri CharacterBase adlı ebeveyn sınıfta toplanmıştır. Bu merkezi yapılandırma sayesinde, her karakter için aynı değişkenlerin tekrar tanımlanması engellenmiş ve kod tekrarının önüne geçilmiştir.

Mimari kurgunun işlevselliği, çok biçimlilik prensibi ile derinleştirilmiştir. Temel sınıftaki Die() ve TakeDamage() gibi metotlar sanal olarak tanımlanarak, PlayerController ve EnemyAI gibi türetilmiş sınıfların bu metotları ezmesi ve kendilerine özgü davranış modellerini uygulaması sağlanmıştır. Bu yapı sayesinde,

aynı temel metot çağrıldığında oyuncu karakteri görsel bir geri bildirim verirken, düşman karakteri ses efekti ile tepki verebilmektedir. Ayrıca, kompozisyon ve modülerlik ilkesi gözetilerek karakter kontrol mekanizmaları tek bir sınıfa yığılmak yerine parçalara ayrılmıştır. Oyuncu girdileri için InputHandler, düşman yapay zekâsı için ise EnemyStateMachine gibi özelleşmiş modüller ilgili sınıflara enjekte edilerek, Tek Sorumluluk ilkesine uygun, yönetilebilir bir yapı elde edilmiştir [7].



Şekil 3.1. Halisler Projesi UML Sınıf Mimarisi

3.2. Fizik Mimarisi ve Karakter Kontrolcüsü

Oyunun kuş bakışı (Top-Down) yapısı, karakter kontrol mekanizmasının geleneksel platform oyunlarından farklılaşarak yerçekimsiz bir vektör uzayında kurgulanmasını zorunlu kılmıştır. Kod tarafında oluşturulan matematiksel modellemede, karakterin hareketi X ve Y eksenleri üzerindeki kuvvetlerin bileşkesi olarak hesaplanmaktadır. Bu hesaplama sürecinde, oyuncunun çapraz hareket etmesi durumunda hipotenüs

etkisiyle oluşabilecek istemsiz hızlanmayı engellemek amacıyla hareket vektörü normalizasyon işlemine tabi tutulmuştur. Bu işlem sayesinde, karakterin her yöne sabit bir hızda ilerlemesi matematiksel olarak garanti altına alınmıştır. Kontrol mekanizmasının tepki süresini optimize etmek için ise `Input.GetAxisRaw` metodu tercih edilmiş; bu sayede tuş hassasiyeti maksimize edilerek, karakterin tuşa basıldığı an harekete geçmesini ve bırakıldığı an durmasını sağlayan keskin bir hareket dinamiği oluşturulmuştur.

Karakterin hesaplanan bu hareket vektörüyle çevreyle kurduğu etkileşim ise Unity'nin fizik motoru (Physics 2D) üzerinden yönetilmektedir. Karakterin duvarların içinden geçmemesi ve çarpışmaları doğru algılayabilmesi için sisteme `Rigidbody2D` bileşeni entegre edilmiştir. Oyunun perspektif yapısı gereği, yerçekimi etkisi (Gravity Scale) 0 olarak ayarlanarak karakterin ekranın altına düşmesi engellenmiş; hızlı hareket sekanslarında nesnelerin iç içe geçmesini önlemek amacıyla da çarpışma algılama modu "Continuous" olarak yapılandırılmıştır.

```

private void Update()
{
    float moveX = inputHandler.GetHorizontalInput();
    float moveY = inputHandler.GetVerticalInput();

    movementInput = new Vector2(moveX, moveY).normalized;

    HandleAnimation(moveX, moveY);
}

private void FixedUpdate()
{
    Move();
}

private void Move()
{
    playerBody.velocity = movementInput * moveSpeed;
}

private void HandleAnimation(float x, float y)
{
    if(movementInput.magnitude > 0)
    {
        playerAnimator.SetFloat("InputX", x);
        playerAnimator.SetFloat("InputY", y);
        playerAnimator.SetBool("IsMoving", true);

        if (x > 0) transform.localScale = new Vector3(1,1,1);
        else transform.localScale = new Vector3(-1,1,1);
    }
    else playerAnimator.SetBool("IsMoving", false);
}

```

Şekil 3.2. PlayerController sınıfı içerisindeki vektörel hareket hesaplaması.

3.3. Animasyon Sistemleri ve Karakter Kontrolcüsü (FSM)

Oyunun görsel geri bildirim döngüsünü oluşturmak ve hedeflenen prototipleme sürecini tamamlamak amacıyla, animasyon sistemi bu aşamada üçüncü parti hazır varlık kütüphaneleri kullanılarak yapılandırılmıştır. Benimsenen bu yaklaşım, nihai görsel tasarım süreçlerine geçilmeden önce hareket mantığının ve animasyon geçişlerinin teknik doğruluğunun test edilmesini sağlamıştır.

3.3.1. Sprite entegrasyonu ve konfigürasyon

Kullanılan piksel tabanlı hazır varlık paketleri, Unity oyun motoruna aktarım sürecinde modern bir piksel oyunun estetik gereksinimlerini karşılayacak keskinlikte

yapılandırılmıştır. Bu amaçla, motorun varsayılan doku ayarları üzerinde spesifik optimizasyonlar gerçekleştirilmiştir. İlk olarak, piksellerde istenmeyen bulanıklığa yol açan varsayılan "Bilinear" filtreleme modu, "Point (No Filter)" seçeneği ile değiştirilerek piksel sanatının karakteristik keskin hatları koruma altına alınmıştır. Görsel bütünlüğün sağlanması ve sıkıştırma kaynaklı artefakt oluşumunun engellenmesi adına format ayarı "RGBA 32 bit" olarak sabitlenmiştir. Entegrasyon sürecinin son aşamasında ise tek bir pafta halinde sisteme dahil edilen animasyon setleri, Unity Sprite Editor aracı kullanılarak ızgara veya otomatik mod seçenekleriyle taranmış ve bağımsız karelere dönüştürülerek kullanıma hazır hale getirilmiştir.

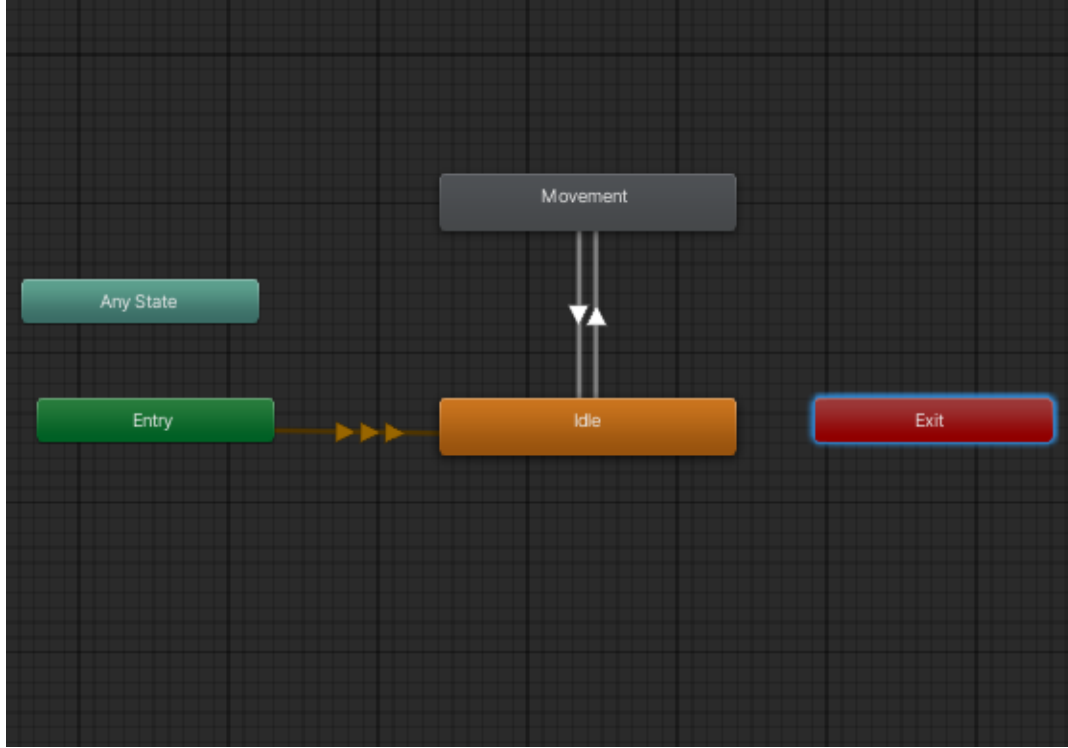


Şekil 3.3. Doku ayarları optimize edilmeden önce ve sonra

3.3.2. Animator kontrolcüsü ve mantıksal geçişler

Karakterin görsel durumları, Unity'nin Animator bileşeni üzerinde kurgulanan Sonlu Durum Makinesi (FSM) mimarisi aracılığıyla yönetilmekte; kod katmanında işlenen hareket verileri görsel katmana anlık olarak iletilmektedir. Oluşturulan animasyon ağacı, oyuncudan herhangi bir girdi gelmediğinde varsayılan olarak aktifleşen "Bekleme" ve karakterin hareket hızı belirli bir eşik değerini aştığında tetiklenen "Koşma" olmak üzere iki temel durumu kapsamaktadır. Durumlar arasındaki bu mantıksal geçişler, PlayerController sınıfından gönderilen hız parametresi referans

alınarak kontrol edilmekte, böylece karakterin durma ve hareket etme eylemleri arasındaki görsel geçiş milisaniyelik hassasiyetle senkronize edilmektedir.



Şekil 3.4. Karakterin Idle ve Run durumları arasındaki geçiş mantığını gösteren Animator şeması.

3.4. Arayüz (UI) Programlama ve Prototipleme

Oyunun anlatı yapısını destekleyen kullanıcı arayüzü (UI) tasarımı ve prototipleme süreçlerinde; oyuncu deneyimini sade ve odaklı tutmak amacıyla Unity Canvas ve TextMeshPro (TMP) teknolojileri kullanılarak oyun dünyasından bağımsız (Non-Diegetic) bir mimari kurgulanmıştır. Farklı ekran çözünürlüklerinde tutarlı bir görünüm sağlamak adına duyarlı bir ölçeklendirme sistemi benimsenmiş; diyalog ve etkileşim mekanikleri ise ekranı görsel ve metin alanı olarak ayıran dinamik bir yerleşim planı üzerine inşa edilmiştir.

3.4.1. UI altyapısı ve teknoloji seçimi

Çalışma kapsamında, oyunun anlatı dinamiklerine hizmet edecek diyalog sistemi ve karakter etkileşimleri, Unity Canvas sistemi ve TextMeshPro (TMP) entegrasyonu

kullanılarak endüstri standartlarına uygun şekilde prototiplenmiştir. Tasarım sürecinde, oyuncu deneyimini sadeleştirmek ve odak noktasını temel anlatı akışında tutmak amacıyla Non-Diegetic (oyun dünyasından bağımsız) arayüz elementleri tercih edilmiştir.

Bu tercihi destekleyen Canvas mimarisinde render modu "Screen Space – Overlay" olarak yapılandırılarak, arayüzün sahne içeriğinden bağımsız bir katman olarak her zaman ekranın en önünde görüntülenmesi sağlanmıştır. Farklı ekran çözünürlüklerinde ve en-boy oranlarında görsel tutarlılığı korumak adına ise Canvas Scaler bileşeninde "Scale With Screen Size" modu etkinleştirilmiş ve referans çözünürlük 1920x1080 olarak sabitlenmiştir. Bu konfigürasyon, projenin farklı cihazlarda sorunsuz çalışabilmesi için kritik öneme sahip olan uyarlanabilir UI altyapısının temelini oluşturmaktadır.

3.4.2. Dinamik diyalog ve etkileşim arayüzü

Dinamik diyalog sistemi için kurgulanan UI altyapısı, kullanıcının metin bazlı akışı takip etmesine ve eşzamanlı olarak klavye üzerinden girdi sağlamasına olanak tanıyacak fonksiyonel bir zeminde tasarlanmıştır. Bu tasarım mimarisinde, ekranı yatay düzlemde iki eşit parçaya bölen simetrik bir yerleşim planı benimsenmiştir. Ekranın sol yarısını tamamen kaplayacak şekilde 960x540 piksel boyutlarında kurgulanan PortraitImage objesi, konuşan karakterin yüksek çözünürlüklü piksel sanat görselini sergilemek üzere sol kenara sabitlenmiştir. Buna karşılık, ekranın sağ yarısına konumlandırılan ve aynı boyutlara sahip DialoguePanel objesi ise etkileşim ve metin akışı için sağ kenara hizalanmıştır.

Arayüzün görsel kimliği, oyunun genel atmosferiyle bütünleşmesi amacıyla kâğıt, ahşap ve savaş ekipmanları temalı özel piksel sanat varlıklarıyla zenginleştirilmiştir. Söz konusu görsellerin sahne içinde netlik kaybı yaşamadan render edilebilmesi için doku ayarlarındaki filtreleme modu "Point (No Filter)" olarak belirlenmiştir. Bu teknik konfigürasyon, modern oyun motorlarının otomatik yumuşatma işlemlerinden kaynaklanan bulanıklık sorununu ortadan kaldırarak piksel sanatının keskin hatlarını korumaktadır.

Panel üzerindeki metin bileşenleri, arka plan görsellerinin piksel sınırlarına tam uyum sağlayacak şekilde Anchor ve Offset değerleri manuel olarak optimize edilerek konumlandırılmıştır. Konuşma dökümünün aktarıldığı Text bileşeni, ana panel görselindeki kâğıt alanına hizalanırken; oyuncunun diyalog girişi yapacağı TMP Input Field bileşeni (KeyboardInput), panelin alt kısmındaki yatay şerit sınırlarına yerleştirilmiştir. Tercih edilen bu giriş bileşeni, metin girişinin işlenmesi ve takip edilmesi süreçlerinde kritik rol oynayan onEndEdit gibi etkileşim fonksiyonlarını doğal olarak destekleyerek teknik altyapıyı güçlendirmektedir.



Şekil 3.5. UI Tasarımı Prototipi

3.5. Oyun Akış Yönetimi (Cinematic Manager)

Oyunun açılış sekanslarında veya kritik hikâye dönüm noktalarında sahne akışını yönetmek üzere CinematicManager adı verilen merkezi bir kontrol mekanizması geliştirilmiştir. Sahnedeki NPC'ler, oyuncu ve kamera sistemleri gibi tüm aktif bileşenlerin durumlarını denetleyen bu yönetici sınıf, sisteme entegre edilen Canvas bileşeni üzerinden görsel sunumu koordine etmektedir. Projedeki öncelikli işlevi başlangıç kontrolünü sağlamak olan sistem, ilk hikâye tetikleyicisini devreye sokarak oyuncunun doğrudan atmosferin içine dahil olmasını hedefler. Sistemin temel çalışma prensibi; oyuncu hareketlerini geçici olarak kısıtlayarak, kurgulanan

animasyon, ses dosyası veya olay örgüsünün kesintisiz bir şekilde oynatılması üzerine kuruludur.

Oyunun başlangıç sekansını ve sinematik geçişini yöneten akış, IEnumerator PlayIntroSequence fonksiyonunun kontrolü ile sağlanır. Bu fonksiyonda; öncelikle atmosfer sesleri senkronize bir şekilde oynatılır, ardından siyah ekran arayüzü kaldırılarak sahne görünür hale getirilir ve son olarak yapay zekâya hikâye bağlamı kod üzerinden enjekte edilerek, oyuncunun girdisine gerek kalmadan açılış diyalogu tetiklenmiş olur.

```
IEnumerator PlayIntroSequence()
{
    yield return new WaitForSeconds(1f);

    foreach(var ses in introSounds)
    {
        if(ses != null)
        {
            audioSource.PlayOneShot(ses);
            yield return new WaitForSeconds(ses.length + 0.5f);
        }
    }

    float duration = 2.0f;
    float timer = 0f;
    while (timer < duration)
    {
        timer += Time.deltaTime;
        blackScreenGroup.alpha = Mathf.Lerp(1f, 0f, timer / duration);
        yield return null;
    }

    blackScreenGroup.alpha = 0;
    blackScreenGroup.interactable = false;
    blackScreenGroup.blocksRaycasts = false;

    if (baslangicNPC != null)
    {
        string gorevBaglami = "Oyunun açılış sahnesi. Kapıyı çaldın ve Efendi İlyas karşına çıktın. Kralın onu çağırdığını söyle, bu konuda başka hiçbir şey bilmiyorsun, büyücü neden çağrıldığını sorarsa bilmiyorum de, sana verilen görev mesajı iletmek. Sakın başka konulara girme";
        baslangicNPC.Talk("", gorevBaglami);
    }
}
```

Şekil 3.6. IEnumerator PlayIntroSequence fonksiyonu

BÖLÜM 4. YAPAY ZEKÂ ENTEGRASYONU VE DİNAMİK ETKİLEŞİM SİSTEMLERİ

Bu bölümde, projenin en yenilikçi bileşeni olan ve geleneksel oyun geliştirme süreçlerinden ayrılan yapay zekâ entegrasyonunun teknik detayları ele alınmaktadır. Statik diyalog ağaçları yerine Büyük Dil Modelleri (LLM) kullanılarak kurgulanan dinamik etkileşim mimarisi, persona mühendisliği ve veri odaklı tasarım (Data-Oriented Design) prensipleri çerçevesinde incelenmiştir. Ayrıca, Unity oyun motoru ile bulut tabanlı yapay zekâ servisleri arasında kurulan asenkron REST API iletişim katmanı, JSON veri işleme algoritmaları ve doğal dil işleme süreçlerinin teknik implementasyonu bu bölümün temel kapsamını oluşturmaktadır.

4.1. AI Model Seçimi ve Persona Mühendisliği

Geleneksel oyunlarda kullanılan statik ve önceden yazılmış diyalog ağaçlarının kısıtlayıcı yapısı yerine, oyuncuya etkileşim özgürlüğü tanıyan ve dinamik hikâye anlatımını destekleyen Büyük Dil Modelleri (LLM) tabanlı bir karakter etkileşim sistemi tasarlanmıştır. Projenin gerçek zamanlı etkileşim gereksinimleri doğrultusunda, yüksek işlem maliyeti ve gecikme süresi yaratan büyük modeller yerine hız ve performans odaklı çözümlere yönelilmiştir. Yapılan API testleri neticesinde; düşük gecikme süresi ile anlık yanıt üretebilmesi, prototipleme aşamasındaki maliyet avantajı ve geniş bağlam penceresi sayesinde konuşma geçmişini tutarlı bir şekilde yönetebilmesi nedeniyle Google Gemini 2.5 Flash modeli projeye entegre edilmiştir.

Yapay zekâ karakterlerinin oyun dünyasıyla bütünleşmesi ve tutarlı kalabilmesi amacıyla, literatürdeki 'Üretken Ajanlar' mimarisi referans alınarak Sistem Talimatları (System Instructions) yöntemi uygulanmıştır. Modelin ham hali üzerine inşa edilen bu katmanlı yapı sayesinde; karaktere spesifik kimlik tanımları, duygusal durumunu belirleyen kişilik özellikleri ve oyunun atmosferine uygun "Eski Türkçe" üslubu enjekte edilmiştir. Ayrıca, yanıtların uzunluğunu sınırlandıran ve yapay zekâ

kimliğini gizleyen katı kısıtlamalar ile simülasyonun inandırıcılığı güvence altına alınmıştır. Teknik mimarinin işleyişi, oyuncu ile NPC arasındaki etkileşimi sürekli kılan bir döngü üzerine kuruludur. Sistem, her yeni oyuncu girdisinde önceki konuşma geçmişini de paketleyerek API'ye iletir; bu sayede karakterler, oyuncunun adını veya geçmiş diyalog detaylarını hatırlayarak bağlamsal bütünlüğü koruyan tutarlı bir iletişim sürdürebilmektedir.

Oyun motoru ile Bulut AI servisi arasındaki veri trafiğini yönetmek amacıyla GeminiManager sınıfı geliştirilmiştir. Singleton tasarım kalıbı ile kurgulanan bu merkezi yönetici, UnityWebRequest kütüphanesini kullanarak HTTP POST protokolü üzerinden haberleşme sağlar. Kodun en kritik fonksiyonu olan SendRequest korutini (Coroutine), ana oyun döngüsünü bloklamadan asenkron olarak çalışır [8]. Bu mimari, JSON formatındaki persona verilerini ve kullanıcı girdilerini paketleyip API'ye iletirken; olası bağlantı kopuklukları veya hız sınırları (Rate Limit - HTTP 429) gibi hata durumlarını da yakalayarak sistemin kararlılığını korur ve oyun akışının kesilmesini engeller.

```

public static GeminiManager Instance;

[Header("API Ayarları")]
public string apiKey = "*****";
private string apiUrl = "https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-
latest:generateContent";

void Awake()
{
    if (Instance == null) Instance = this;
    else Destroy(gameObject);
}

public void ChatWithGemini(string persona, string userMessage, Action<string> onComplete)
{
    StartCoroutine(SendRequest(persona, userMessage, onComplete));
}

IEnumerator SendRequest(string persona, string userMessage, Action<string> callback)
{
    string cleanUrl = apiUrl.Trim();
    string cleanKey = apiKey.Trim();
    string fullUrl = $"{cleanUrl}?key={cleanKey}";

    Debug.Log("Bağlanılıyor: " + fullUrl);

    string finalPrompt = $"{persona}\n\nOyuncu: \"{userMessage}\".\n(Kısa cevap ver)";
    string jsonData = "{\"contents\": [{\"parts\": [{\"text\": \"" + EscapeJson(finalPrompt) +
    "\"]}]}";
    using (UnityWebRequest request = new UnityWebRequest(fullUrl))
    {
        byte[] bodyRaw = Encoding.UTF8.GetBytes(jsonData);

        request.uploadHandler = new UploadHandlerRaw(bodyRaw);
        request.downloadHandler = new DownloadHandlerBuffer();

        request.method = UnityWebRequest.kHttpVerbPOST;

        request.SetRequestHeader("Content-Type", "application/json");

        yield return request.SendWebRequest();

        if (request.result == UnityWebRequest.Result.Success)
        {
            string responseJson = request.downloadHandler.text;

            try
            {
                GeminiResponse response = JsonUtility.FromJson<GeminiResponse>(responseJson);
                if (response != null && response.candidates != null && response.candidates.Length >
0)
                {
                    string aiReply = response.candidates[0].content.parts[0].text;
                    callback?.Invoke(aiReply);
                }
                else
                {
                    Debug.LogError("Cevap boş geldi veya çözülemedi: " + responseJson);
                }
            }
            catch (Exception e)
            {
                Debug.LogError("JSON Hatası: " + e.Message);
            }
        }
        else
        {
            // DETAYLI HATA RAPORU
            Debug.LogError($"HATA KODU: {request.responseCode}");
            Debug.LogError($"HATA MESAJI: {request.error}");
            Debug.LogError($"GOOGLE DÖNÜŞÜ: {request.downloadHandler.text}");

            if (request.responseCode == 429)
                callback?.Invoke("(Nefes nefese)* Çok hızlı sordun, 1 dakika bekle...");
            else if (request.responseCode == 404)
                callback?.Invoke("API Adresli veya Model bulunamadı. Key veya URL hatalı.");
            else
                callback?.Invoke("... (Bağlantı hatası)");
        }
    }
}
}

```

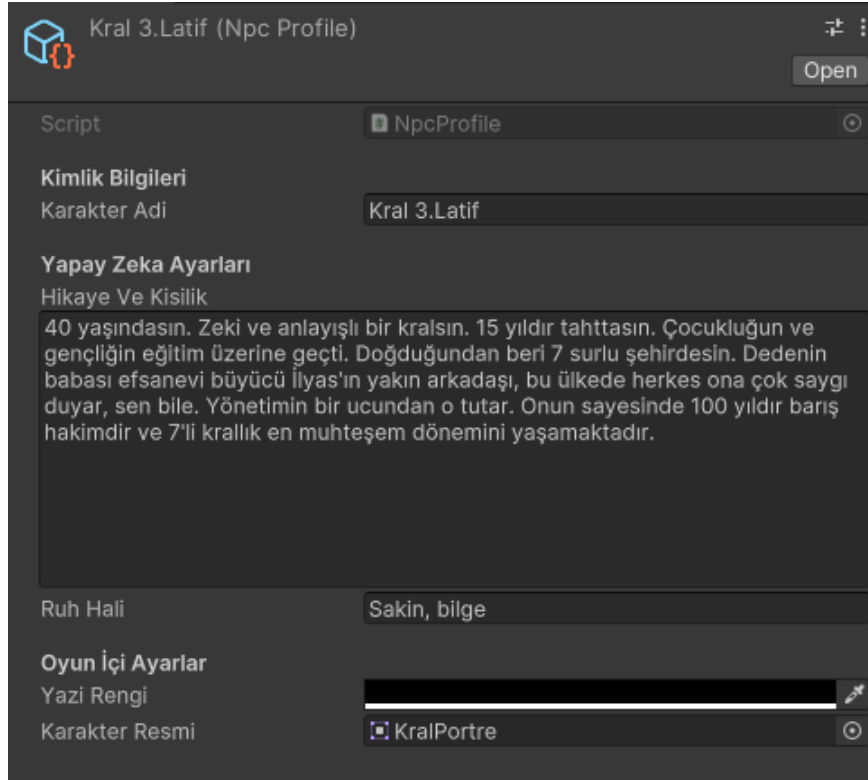
Şekil 4.1. GeminiManager sınıfı ve veriyi API'ye gönderen SendRequest fonksiyonu

4.2. Modüler Npc Mimarisi

Bu bölüm, projenin inovasyon merkezini oluşturan "Yapay Zekâ Entegreli NPC" sisteminin teknik altyapısını detaylandırmaktadır. Mimari kurguda, geleneksel oyun geliştirme süreçlerinde yaygın olan ve her karakter için ayrı kod yazılmasını gerektiren yaklaşım terk edilmiştir. Bunun yerine, sistemin ölçeklenebilirliği ve bakım kolaylığı gözetilerek Veri Odaklı Tasarım (Data-Oriented Design) prensipleri benimsenmiş; Unity'nin ScriptableObject teknolojisi temel alınarak esnek ve modüler bir yapı inşa edilmiştir.

4.2.1. Veri kapsayıcıları

Oyun evrenindeki karakterlerin (Kral, Asker vb.) kişilik özellikleri ve görsel kimliklerini kod yapısından bağımsız hale getirmek amacıyla NpcProfile adında özelleştirilmiş bir veri sınıfı geliştirilmiştir. Unity'nin ScriptableObject mimarisinden türetilen bu sınıf; veriyi oyun çalışma zamanından bağımsız olarak disk üzerinde saklayabilen ve bellekte tek bir referans kopyası üzerinden yönetilen yüksek verimli bir veri konteyneri işlevi görmektedir. Bu mimari tercih sayesinde oyun tasarımcıları, herhangi bir kodlama işlemine ihtiyaç duymadan doğrudan editör arayüzü üzerinden NPC kimliklerini, persona tanımlarını ve görsel referanslarını yapılandırabilmektedir. Sistemin sağladığı bu veri soyutlaması, projeye yüzlerce farklı NPC eklendiği senaryolarda dahi bellek yönetiminin optimize edilmesini ve performans kararlılığının korunmasını garanti altına almaktadır.



Şekil 4.2. NpcProfile veri yapısının Unity Inspector üzerindeki görünümü

4.2.2. Sahne içi kimlik yönetimi

Soyut veri yapılarının (NpcProfile) oyun sahnesindeki somut temsilini sağlamak ve fiziksel varlıklarla ilişkilendirmek üzere NpcIdentity sınıfı geliştirilmiştir. Sahneye yerleştirilen her NPC objesine bileşen olarak entegre edilen bu sınıf, statik veri dosyaları ile dinamik oyun dünyası arasında işlevsel bir köprü görevi görmektedir. Çalışma prensibi olarak NpcIdentity; kendisine atanan profil dosyasını referans olarak karakterin kimlik bilgilerini, mevcut durumunu ve ruh halini tanımlar. Sistemin iletişim düğümü olarak hareket eden bu yapı; oyuncudan veya çevresel tetikleyicilerden gelen girdileri karşılar, bu verileri karakterin profil parametreleriyle sentezler ve işlenmek üzere yapay zekâ yöneticisine iletir. Kurgulanan bu modüler mimari sayesinde, tek bir kod tabanı farklı karakter arketipleri (Kral, Asker vb.) üzerinde ortak olarak kullanılabilen; karakter davranışları sadece atanan profil dosyalarının değiştirilmesiyle dinamik ve esnek bir şekilde yönetilebilmektedir.

Tasarlanan persona mühendisliği ve bağlam yönetimi, Unity tarafında Talk fonksiyonu aracılığıyla işlevsel hale getirilmiştir. Bu fonksiyon, oyunun o anki

durumunu, karakterin statik kişiliğini ve dinamik görev bağlamını birleştirerek, Büyük Dil Modeli'ne (LLM) gönderilecek nihai "Sistem İstemi"ni çalışma zamanında inşa eder.

Karakterin profilDosyasi üzerinden gelen kimlik bilgileri, context parametresi ile gelen anlık görev bilgisi ve sistemin halüsinasyon görmesini engelleyen "Kesin Kurallar" tek bir metin bloğunda birleştirilmektedir. Ayrıca, LLM'in cevabı içerisinde [SON] etiketinin taranması, yapay zekânın konuşmayı ne zaman bitireceğine karar vermesini sağlayan bir oturum sonlandırma protokolü olarak kurgulanmıştır.

```

public void Talk(string playerMessage = "", string context = "")
{
    if (profilDosyasi == null) return;

    DialogueUIManager.Instance.SetActiveNpc(this);
    DialogueUIManager.Instance.ShowThinking(profilDosyasi.karakterAdi, profilDosyasi.yaziRengi,
    profilDosyasi.karakterResmi);

    string systemPrompt = $"Senin Adın: {profilDosyasi.karakterAdi}.\n" +
        $"Kişiliğin: {profilDosyasi.hikayeVeKisilik}.\n" +
        $"Ruh Halin: {profilDosyasi.ruhHali}.\n";

    if (!string.IsNullOrEmpty(context))
        systemPrompt += $"[GÖREVİN VE BİLDİKLERİN]: {context}.\n";

    systemPrompt += "\n[KESİN KURALLAR - BUNLARI ASLA ÇİĞNEME]:\n" +
        "1. Sadece yukarıdaki 'GÖREVİN VE BİLDİKLERİN' kısmında yazanları biliyorsun.\n"
        +
        "2. Eğer oyuncu sana bu bilgiler dışında bir şey sorarsa (Neden, Nasıl, Kim vb.)
        ve cevabı metinde yoksa ASLA SENARYO UYDURMA.\n" +
        "3. Bilmediğin konularda 'Bilmiyorum', 'Bana sadece bu kadar söylendi', 'Emir
        kuluyum' gibi cevaplar ver.\n" +
        "4. ASLA oyuncuya hayranlık duyma, aşık olma veya samimi olma (Kişiliğinde
        yazmıyorsa).\n";

    string finalMessageToSend = "";

    if (string.IsNullOrEmpty(playerMessage))
    {
        systemPrompt += "GÖREV: Konuşmayı başlat (Max 15 kelime).\n";
        finalMessageToSend = "Konuşmayı başlat.";
    }
    else
    {
        systemPrompt += $"Oyuncu dedi ki: \"{playerMessage}\".\n";

        systemPrompt += "KURAL: Eğer oyuncu gitmek isterse ya da saçma bir şey söylerse, 'Tamam' de
        ve cümlenin sonuna [SON] yaz.\n";

        finalMessageToSend = playerMessage;
    }

    systemPrompt += "Cevabın kısa olsun (Max 30 kelime).";

    GeminiManager.Instance.ChatWithGemini(systemPrompt, finalMessageToSend, (gelenCevap) =>
    {
        bool sohbetBittiMi = false;
        if (gelenCevap.Contains("[SON]"))
        {
            sohbetBittiMi = true;
            gelenCevap = gelenCevap.Replace("[SON]", "").Trim();
        }

        DialogueUIManager.Instance.ShowDialogue(profilDosyasi.karakterAdi, gelenCevap,
        profilDosyasi.yaziRengi, profilDosyasi.karakterResmi);

        if (sohbetBittiMi) DialogueUIManager.Instance.EndConversation(2.0f);
    });
}

```

Şekil 4.3. NPC'ye profilini yükleyen ve duruma göre cevap üretmesini sağlayan Talk fonksiyonu

4.3. REST API İletişim Katmanı ve Veri İşleme

Unity oyun motoru ile Google'ın bulut tabanlı yapay zekâ servisi (Gemini AI) arasında kurulan veri köprüsünün teknik mimarisi ve iletişim protokolleri, sistem entegrasyonunun temelini oluşturmaktadır. Bu süreç, oyun döngüsünün

performansını ve akıcılığını korumak amacıyla, ana iş parçacığından bağımsız çalışan asenkron bir yapı üzerine inşa edilmiştir [9].

4.3.1. API mimarisi

Tüm yapay zekâ iletişim süreçlerinin merkezi bir yapı üzerinden yönetilmesi ve sahne genelinde erişilebilirliğin sağlanması amacıyla GeminiManager sınıfı geliştirilmiştir. Bellek yönetimi ve tutarlılık esasları çerçevesinde Singleton Tasarım Deseni uygulanarak, çalışma zamanı boyunca sınıfın tek bir örneğinin (instance) aktif kalması garanti edilmiştir. Söz konusu yönetici katman, NPC etkileşimlerinden elde edilen ham metin verilerini ve sistem talimatlarını işleyerek, Google API standartlarına uygun JSON formatına dönüştürmektedir. Veri güvenliği kapsamında API anahtarları istek başlıklarına entegre edilmekte ve iletim süreci, Unity'nin UnityWebRequest kütüphanesi aracılığıyla HTTP POST protokolü üzerinden gerçekleştirilmektedir.

4.3.2. Optimizasyon Süreci

Projenin geliştirme sürecinde, API yanıt süreleri ve servis kararlılığı üzerine kapsamlı performans analizleri gerçekleştirilmiştir. Başlangıçta hedeflenen mimaride karşılaşılan "429 Too Many Requests" (Aşırı İstek) ve "503 Service Unavailable" (Servis Kullanılamıyor) hata kodları, model entegrasyon stratejisinin yenilenmesini zorunlu kılmıştır. Yürütülen test süreçleri neticesinde, simülasyon ortamı için en optimize modelin gemini-flash-latest sürümü olduğu tespit edilmiştir. Söz konusu modelin tercih edilmesinde; yoğun diyalog döngülerinde kota aşımı riskini minimize etmesi ve ortalama 1-2 saniyelik yanıt süresi ile oyun akıcılığını koruması belirleyici faktör olmuştur.



Şekil 4.4. API testlerinde kota sınırının aşılmasıyla oluşan hata durumu

4.3.3. JSON veri işleme

API servisinden elde edilen çok katmanlı JSON veri yapısının, ham haliyle simülasyon ortamına entegrasyonu mümkün değildir. Bu bağlamda, veri setini işleyerek yalnızca anlamsal içerik bloğunu izole eden özelleştirilmiş bir ayrıştırma algoritması geliştirilmiştir [10]. Veri akış yönetiminde C# Action/Callback (Geri Çağırma) mekanizması esas alınmıştır. Ağ gecikmelerinden kaynaklanabilecek performans kayıplarını ve ana iş parçacığının kilitlenmesini önlemek amacıyla, işlem süreçleri Unity'nin Coroutine mimarisi üzerinde asenkron olarak yürütülmektedir. Sunucudan yanıt döndüğünde Callback fonksiyonu tetiklenmekte ve işlenen metin verisi, senkronizasyon sağlanarak Diyalog Yöneticisi modülüne iletilmektedir.

4.4. Diyalog Yönetimi ve Doğal Dil İşleme

Arka planda çalışan yapay zekâ mimarisinin oyuncuya bakan yüzü olan bu katman, teknik süreçlerin görsel bir deneyime dönüştürüldüğü noktadır. Bu bölümde; dinamik UI elementlerini kontrol eden **DialogueUIManager** sınıfı ve Olay Güdümlü Programlama prensibiyle işlenen kullanıcı girdilerinin, arayüzden mantık katmanına aktarıldığı veri akış mimarisi ele alınmaktadır.

4.4.1. Diyalog arayüzü yöneticisi

Arka plan mimarisinin sunum katmanı olarak görev yapan DialogueUIManager, yapay zekâ tabanlı metin verilerini işleyerek oyuncu için görsel bir deneyime dönüştüren merkezi yönetici sınıftır. Bu sınıf, diyalog süreçlerinin yaşam döngüsünü (panel açma/kapama) yönetirken; konuşmacıya ait isim, renk kodu ve portre görseli gibi kimlik verilerini NpcProfile kaynağından çekerek arayüze dinamik olarak entegre eder. Ayrıca, etkileşim sırasında oyun akışını stabilize etmek amacıyla "Input Kilidi" mekanizmasını devreye sokarak, diyalog boyunca karakterin harici aksiyon almasını engeller.



Şekil 4.5. Diyalog Arayüzü Yöneticisinin çalışma anı

4.4.2. Kullanıcı girdisinin işlenmesi

Diyalog sisteminin etkileşim omurgasını, oyuncunun serbest metin girişi yapabildiği TMP_InputField bileşeni oluşturmakta olup; süreç Olay Güdümlü Programlama prensipleriyle yönetilmektedir. Veri akışı, oyuncunun metin girişini tamamlayıp "Enter" tuşuyla onaylaması ve onEndEdit olayının tetiklenmesiyle başlar. DialogueUIManager bu olayı dinleyerek ham veriyi yakalar, boş değer kontrolünden geçirir ve mükerrer etkileşimi önlemek adına giriş sistemini kilitler. Doğrulanmış veri, NpcIdentity sınıfındaki Talk fonksiyonuna parametre olarak iletilir. Bu fonksiyon; o

anki oyun durumunu, karakterin profil verilerini ve oyuncu mesajını tek bir veri paketi haline getirerek işlenmek üzere GeminiManager servisine teslim eder. Kurgulanan bu zincirleme yapı (UI → NPC → Manager); arayüz katmanı ile iş mantığını birbirinden izole ederek "Temiz Kod" (Clean Code) standartlarına uygun, modüler bir mimari sunar.



Şekil 4.6. Kullanıcı girdisine ("Adın ne asker?") karşılık üretilen, karakter kimliğine uygun dinamik yapay zekâ yanıtı.

BÖLÜM 5. SONUÇLAR VE ÖNERİLER

Bu çalışma kapsamında, geniş ölçekli "Halisler" oyun projesinin temel mimari iskeleti kurgulanmış; projenin inovasyon merkezini temsil eden yapay zekâ entegreli NPC sistemi başarıyla prototiplenerek somut bir teknik altyapı oluşturulmuştur. Teorik tasarım prensiplerinin pratik uygulama sahasına taşındığı bu süreçte, modern oyun geliştirme döngüsünün tüm katmanları uçtan uca ele alınarak projenin sürdürülebilir geliştirme zemini hazırlanmıştır.

5.1. Proje Çıktılarının Değerlendirilmesi

Proje geliştirme sürecinde, karmaşık oyun mekaniklerinin modüler ve ölçeklenebilir bir yazılım mimarisine dönüştürülmesi ile proje yönetim süreçlerinin teknik uygulanabilirliği somutlaştırılmıştır. Görsel tasarım aşamasında, 2D piksel sanat estetiğinde derinlik algısı oluşturmak amacıyla Tilemap sistemi ve "Custom Axis Sorting" algoritmaları entegre edilmiş, hedeflenen görsel bütünlük sağlanmıştır. Oyun Dünyası (World Space) ile Ekran Arayüzü (Screen Space) arasındaki etkileşim, kullanıcı eylemlerine duyarlı dinamik UI elementleri ile kurgulanmıştır. Projenin temel hipotezi olan "Oyun motorlarına dış kaynaklı yapay zekâ entegrasyonu" konusu, Unity ve Google Gemini servisi arasında kurulan asenkron veri köprüsü sayesinde doğrulanmış ve çalışan bir prototip ortaya konulmuştur.

5.2. Teknik Performans ve API Analizi

Proje kapsamında gerçekleştirilen teknik analizler ve stres testleri, Genel Amaçlı Dil Modellerinin (LLM API) gerçek zamanlı oyun motorlarına doğrudan entegrasyonunun getirdiği yapısal kısıtlamaları net bir şekilde ortaya koymuştur. Ticari sürdürülebilirlik açısından tespit edilen en kritik darboğaz, bulut tabanlı servislerin yanıt sürelerinde yaşanan dalgalanmaların, milisaniye hassasiyeti

gerektiren oyun akışında senkronizasyon sorunlarına yol açması ve oyuncu deneyimini kesintiye uğratmasıdır.

Performansı negatif etkileyen bir diğer faktör ise modellerin "durumsuz" mimarisidir; bu yapı, diyalog tutarlılığını korumak adına geçmiş konuşma verilerinin her istekte tekrar gönderilmesini zorunlu kılarak veri yükünü ve işlem maliyetini artırmaktadır. Ayrıca, yoğun etkileşim senaryolarında standart API paketlerinin hız sınırlarına takılması, sistemin çoklu kullanıcı ortamlarında ölçeklenebilirliğini kısıtlamaktadır.

Elde edilen tüm bu veriler ışığında; kesintisiz ve yüksek performanslı bir simülasyon deneyimi için genel amaçlı bulut API'leri yerine, oyunun yerel kaynaklarında çalışabilecek veya oyun evrenine özel olarak "İnce Ayar" işleminden geçirilmiş optimize modellerin kullanılması gerektiği sonucuna varılmıştır.



Şekil 5.1. Bağlam kaybı nedeniyle NPC'nin oyuncuyu tanıyamaması ve diyalog bütünlüğünün bozulması

KAYNAKLAR

- [1] CONCERNEDAPE, "Stardew Valley", Steam Mağaza Sayfası.
- [2] UDONIS BLOG, "Gaming Industry Report 2025: Market Size & Trends".
- [3] MORDOR INTELLIGENCE, "Indie Game Market Size, Growth Forecast, Demand & Trends".
- [4] CD PROJEKT RED, "The Witcher 3: Wild Hunt", Steam Mağaza Sayfası.
- [3] FAGERHOLT, E., LORENTZON, M., "Beyond the HUD: User Interfaces for Increased Player Immersion in FPS Games", Chalmers University of Technology, Master Thesis, 2009.
- [4] UNITY TECHNOLOGIES, "Unity User Manual: 2D Physics & Tilemap Systems".
- [5] NYSTROM, R., "Game Programming Patterns: Singleton & State Patterns"
- [8] MICROSOFT, "C# Programming Guide: Asynchronous Programming with async and await"
- [9] GOOGLE AI FOR DEVELOPERS, "Gemini API Documentation & Pricing"
- [10] NEWTONSOFT, "Json.NET Documentation: Serializing and Deserializing JSON"

ÖZGEÇMİŞ

Hasan Yasir Arslan, 30.10.2004 tarihinde Denizli'de doğmuştur. İlkokul eğitimini Karabük'te, ortaokul ve lise eğitimini Denizli'de tamamlamıştır. 2022 yılında Sakarya Üniversitesi Bilgisayar Mühendisliği bölümünü kazanmış olup halen 4. sınıf öğrencisi olarak lisans eğitimine devam etmektedir. 2025 yılında Çemsan Endüstriyel Proses ve Otomasyon San. Tic. A.Ş. firmasında donanım stajını, Alix Inc. (ABD) firmasında ise yazılım stajını başarıyla tamamlamıştır.

Yavuz Selim Ateş, 8 Eylül 2003 tarihinde doğmuştur. İlkokul ve ortaokul eğitimini farklı şehirlerde, lise eğitimini İstanbul Nevzat Ayaz Anadolu Lisesi'nde tamamlamıştır. 2022 yılında Sakarya Üniversitesi Bilgisayar Mühendisliği Bölümü'nü kazanmış olup halen 4. sınıf öğrencisi olarak lisans eğitimine devam etmektedir. 2023 yılında Motion Blur firmasında, 2025 yılında Özyeğin Üniversitesi'nde ve Rosca Teknoloji Yazılım A.Ş. firmasında staj yapmıştır.

BSM 401 BİLGİSAYAR MÜHENDİSLİĞİ TASARIMI DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI

KONU : Hikâye Tabanlı 2D Piksel Oyun: Halisler

ÖĞRENCİLER (Öğrenci No/AD/SOYAD): B221210096/Hasan Yasir/ARSLAN

G221210055/Yavuz Selim/ATEŞ

Değerlendirme Konusu	İstenenler	Not Aralığı	Not
Yazılı Çalışma			
Çalışma klavuza uygun olarak hazırlanmış mı?		0-5	
Teknik Yönden			
Problemin tanımı yapılmış mı?		0-5	
Geliştirilecek yazılımın/donanımın mimarisini içeren blok şeması (yazılımlar için veri akış şeması (dfd) da olabilir) çizilerek açıklanmış mı?			
Blok şemadaki birimler arasındaki bilgi akışına ait model/gösterim var mı?			
Yazılımın gereksinim listesi oluşturulmuş mu?			
Kullanılan/kullanılması düşünülen araçlar/teknolojiler anlatılmış mı?			
Donanımların programlanması/konfigürasyonu için yazılım gereksinimleri belirtilmiş mi?			
UML ile modelleme yapılmış mı?			
Veritabanları kullanılmış ise kavramsal model çıkarılmış mı? (Varlık ilişki modeli, noSQL kavramsal modelleri v.b.)			
Projeye yönelik iş-zaman çizelgesi çıkarılarak maliyet analizi yapılmış mı?			
Donanım bileşenlerinin maliyet analizi (prototip-adetli seri üretim vb.) çıkarılmış mı?			
Donanım için gerekli enerji analizi (minimum-uyku-aktif-maksimum) yapılmış mı?			
Grup çalışmalarında grup üyelerinin görev tanımları verilmiş mi (iş-zaman çizelgesinde belirtilebilir)?			
Sürüm denetim sistemi (Version Control System; Git, Subversion v.s.) kullanılmış mı?			
Sistemin genel testi için uygulanan metotlar ve iyileştirme süreçlerinin dökümü verilmiş mi?			
Yazılımın sızma testi yapılmış mı?			
Performans testi yapılmış mı?			
Tasarımın uygulamasında ortaya çıkan uyumsuzluklar ve aksaklıklar belirtilerek çözüm yöntemleri tartışılmış mı?			
Yapılan işlerin zorluk derecesi?		0-25	
Sözlü Sınav			
Yapılan sunum başarılı mı?		0-5	
Soruları yanıtlama yetkinliği?		0-20	
Devam Durumu			
Öğrenci dönem içerisindeki raporlarını düzenli olarak hazırladı mı?		0-5	
Diğer Maddeler			
Toplam			

DANIŞMAN: Dr. Öğr. Üyesi Kayhan Ayar

DANIŞMAN İMZASI: